

# Aperture — Testing Strategy

**Version:** v1.0

**Document Type:** Testing Strategy

## Related Documents

- Backend Architecture
  - Frontend Architecture
  - API Specification
  - CI/CD Guide (Future)
- 

## 1. Purpose

This document defines the testing philosophy, scope, and standards for Aperture.

The goal is to build confidence in the application by validating functionality at multiple levels rather than relying on a single testing approach.

Testing should provide fast feedback during development while ensuring production reliability.

---

## 2. Testing Philosophy

Testing follows four guiding principles:

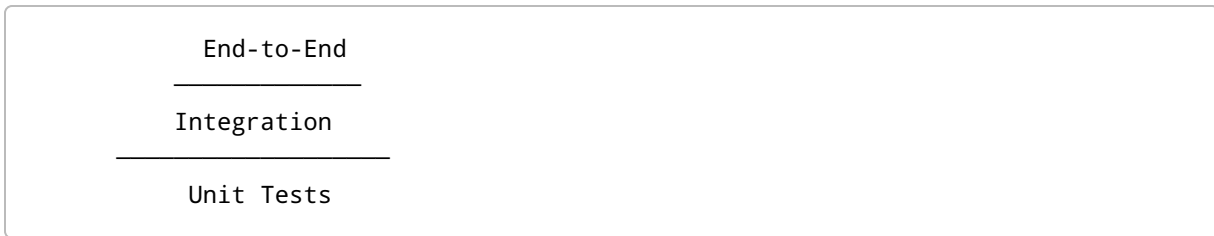
- Test behavior instead of implementation
- Prefer many fast tests over a few slow tests
- Automate everything that can be automated
- Every bug should result in a new test

Tests are part of the product—not an afterthought.

---

## 3. Testing Pyramid

Aperture follows the classic testing pyramid.



Most tests should be unit tests.

Integration tests validate module interactions.

End-to-end tests verify critical user journeys.

---

## 4. Unit Testing

### Scope

Unit tests validate isolated business logic.

Typical targets:

- Services
- Utility functions
- Validation rules
- Ranking algorithms
- Recommendation scoring
- Permission checks

Infrastructure concerns should be mocked where appropriate.

---

### Examples

Authentication

- Password validation
- Token generation
- Refresh token rotation

Reviews

- Duplicate review prevention
- Rating validation
- Spoiler handling

Discovery

- Ranking strategy
  - Diversity logic
  - Feed assembly
- 

## 5. Integration Testing

Integration tests validate interactions between components.

Examples:

- Service ↔ Repository
- API ↔ Database
- Metadata sync ↔ TMDb client
- Search ↔ Metadata
- Notifications ↔ Event handlers

These tests use a real database configured specifically for testing.

---

## 6. API Testing

Every public endpoint should be exercised.

Validation includes:

- Success responses
- Validation failures
- Authentication
- Authorization
- Error responses
- Pagination
- Filtering

The API contract should remain stable across releases.

---

## 7. Frontend Testing

Frontend testing focuses on behavior.

## Component Tests

Examples:

- Button interactions
  - Form validation
  - Dialog behavior
  - Loading states
- 

## Hook Tests

Examples:

- Custom hooks
  - State transitions
  - Query invalidation
- 

## Accessibility Tests

Verify:

- Keyboard navigation
  - ARIA attributes
  - Focus management
  - Contrast issues (where automated tools can help)
- 

# 8. End-to-End Testing

E2E tests validate complete user workflows.

Critical journeys include:

- User registration
- Login
- Search
- Review creation
- List creation
- Follow another user
- Browse recommendations

These tests should be stable, deterministic, and limited to the application's highest-value flows.

---

## 9. Performance Testing

Performance tests measure system behavior under load.

Scenarios include:

- Homepage generation
- Search latency
- Review creation throughput
- Metadata synchronization
- Concurrent users

Performance regressions should be tracked over time.

---

## 10. Security Testing

Security testing validates common attack scenarios.

Examples:

- SQL injection
- XSS prevention
- CSRF (where applicable)
- Authentication bypass
- JWT tampering
- Authorization checks

Security tests should be included in CI where practical.

---

## 11. Test Data Strategy

Test data should be:

- Deterministic
- Isolated
- Easy to understand

Avoid sharing mutable test fixtures between unrelated tests.

Factories or builders should generate realistic domain objects.

---

## 12. Test Environments

Separate environments should exist for:

- Local development
- Continuous Integration
- Staging

Production data should never be used during automated testing.

---

## 13. Mocking Strategy

Mock external dependencies only.

Examples:

- TMDb API
- Email delivery
- OAuth providers
- Push notification services

Avoid mocking business logic within the application.

---

## 14. Continuous Testing

Every pull request should automatically execute:

- Formatting
- Linting
- Type checking
- Backend unit tests
- Frontend tests
- Integration tests
- API tests

A merge should be blocked if required quality gates fail.

---

## 15. Code Coverage

Coverage is a useful metric but not the objective.

Prioritize testing:

- Critical business workflows
- Security-sensitive code
- Complex logic

High coverage with poor assertions provides little value.

Coverage trends should be monitored rather than optimized blindly.

---

## 16. Regression Testing

Every production bug should result in:

1. A failing test
2. A fix
3. A passing regression test

This prevents the same issue from reappearing.

---

## 17. Test Organization

Tests should mirror the application structure.

```
tests/  
├─ unit/  
├─ integration/  
├─ api/  
├─ e2e/  
├─ performance/  
└─ fixtures/
```

Feature-specific tests should remain close to their corresponding modules whenever practical.

---

## 18. CI Integration

Testing integrates directly with the CI pipeline.

Typical workflow:

Developer Push

↓

Lint

↓

Type Check

↓

Unit Tests

↓

Integration Tests

↓

Frontend Tests

↓

Build

↓

Deploy Preview

Only successful pipelines are eligible for merge.

---

## 19. Interview Discussion Points

Be prepared to explain:

- Why follow the testing pyramid?
  - Why mock external services but not business logic?
  - How do integration tests differ from API tests?
  - What deserves an end-to-end test?
  - Why isn't code coverage alone a good quality metric?
  - How does automated testing support continuous deployment?
-



## 20. Summary

Aperture's testing strategy emphasizes confidence through layered validation.

By combining unit, integration, API, frontend, end-to-end, performance, and security testing, the project remains reliable while allowing rapid iteration and refactoring.

Testing is treated as an integral part of software engineering rather than a final verification step, ensuring that the platform remains maintainable as both the codebase and feature set continue to grow.